Pokémon Wordle: Intelligent Guessing Strategies

An Interactive Game and Algorithmic Strategy Comparison

Academic Year 2026–27

Team Members

Badagoni Sri Sahasra (B24DS003)

Malraju Srinayana (B24CS024)

Nagineni Thanmaie (B24EC019)

Parinitha Pulipati (B24CS037)

V. Varshitha (B24ME050)

1. Project Overview

Pokémon Wordle is an interactive guessing game inspired by the Wordle concept. The objective is to identify a hidden Pokémon from a dataset using a limited number of guesses.

The system combines an interactive gameplay environment with intelligent algorithms that attempt to identify the hidden Pokémon efficiently.

The project implements four guessing strategies:

- **Random:** Selects guesses randomly.
- **Filter:** Eliminates candidates using previous feedback.
- **Entropy:** Selects guesses based on expected information gain.
- **Minimax:** Selects guesses to minimize the worst-case remaining candidate set.

The application provides three interaction modes: Human Gameplay, AI Demonstration, and Algorithm Comparison. This allows users to play manually, observe automated decision-making, and evaluate the performance of different strategies.

2. Dataset and Game Design

2.1 Dataset Description

The game uses a dataset containing 386 Pokémon records. Each record includes six attributes used for identification and comparison.

Table 1: Pokémon Dataset Attributes

Attribute	Description
Name	Pokémon name
Generation	Generation to which the Pokémon belongs
Type 1	Primary Pokémon type
Type 2	Secondary Pokémon type, if available
Height	Pokémon height
Weight	Pokémon weight

These attributes provide the information required to compare a guess with the hidden Pokémon and generate feedback.

2.2 Game Rules

The game follows a fixed-guess structure:

- One Pokémon is randomly selected as the hidden answer.
- The player or AI submits a Pokémon guess.
- The system evaluates the guess using the available attributes.
- Feedback is used to guide subsequent guesses.

- A maximum of eight guesses is allowed per game.
- The game ends when the correct Pokémon is identified or all eight attempts are exhausted.

The feedback system compares generation, Pokémon types, height, and weight. Numerical attributes use a 10% tolerance band.

3. Guessing Strategies

Four algorithms are implemented to investigate how different guess-selection approaches affect game performance.

3.1 Random Strategy

The Random strategy selects a Pokémon randomly from the available unguessed Pokémon. It does not use information from previous feedback to optimize subsequent guesses.

This strategy serves as the baseline for evaluating the effectiveness of intelligent guessing methods.

3.2 Filter Strategy

The Filter strategy maintains a set of possible Pokémon. After every guess, it evaluates the feedback and eliminates candidates that do not satisfy the observed attribute constraints.

The next guess is selected randomly from the remaining candidates. This progressively reduces the search space.

3.3 Entropy Strategy

The Entropy strategy selects guesses based on expected information gain.

For a candidate guess, possible feedback outcomes divide the remaining Pokémon into groups. The strategy evaluates the uncertainty of these outcomes and prefers guesses that provide greater information.

The entropy of a probability distribution is:

$$H(X) = -\sum_{i} p_i \log_2(p_i)$$

where p_i represents the probability of an outcome.

3.4 Minimax Strategy

The Minimax strategy evaluates possible feedback partitions and selects the guess that minimizes the size of the largest remaining candidate group.

Its objective can be represented as:

$$g^* = \arg\min_{g} \max_{r} |S_{g,r}|$$

where:

- g represents a possible guess.
- r represents a possible feedback outcome.

- $S_{g,r}$ represents the candidates consistent with that outcome.

This strategy focuses on reducing the worst-case search space rather than maximizing average information gain.

4. System Architecture and Implementation

The project follows a modular architecture in which different responsibilities are handled by separate Python modules.

The major components are:

- **Dataset Loading:** Loads and prepares Pokémon records for gameplay.
- **Game Mechanics:** Manages hidden answers, guesses, feedback generation, and game termination.
- **Solver Modules:** Implement Random, Filter, Entropy, and Minimax strategies.
- **Benchmarking:** Executes automated games and collects performance measurements.
- **Streamlit Interface:** Provides the interactive application and visualizes game progress and results.

This modular design separates game logic, algorithmic decision-making, benchmarking, and user interaction, making the application easier to maintain and extend.

5. Application Interface

The application is developed using Streamlit and provides three main modes of interaction.

5.1 Human Gameplay

Human Gameplay allows users to select Pokémon guesses manually. The application displays feedback after each attempt and tracks the remaining guesses.

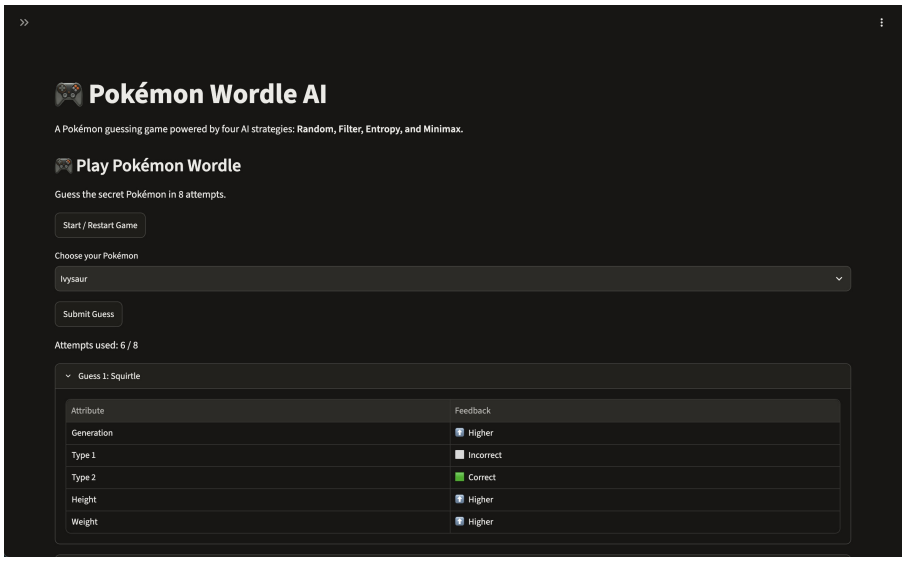


Figure 1: Human Gameplay Interface

5.2 AI Demonstration

AI Demonstration allows users to observe an automated guessing strategy. The selected algorithm makes guesses and uses feedback to determine subsequent choices.

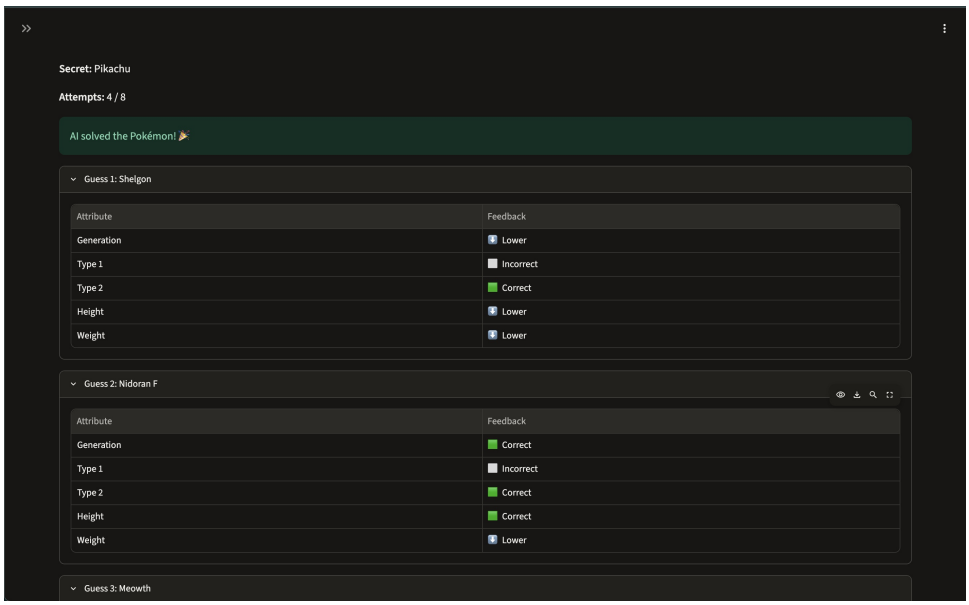


Figure 2: AI Demonstration Interface

5.3 Algorithm Comparison

Algorithm Comparison provides a way to examine the performance of the implemented strategies using benchmark results.

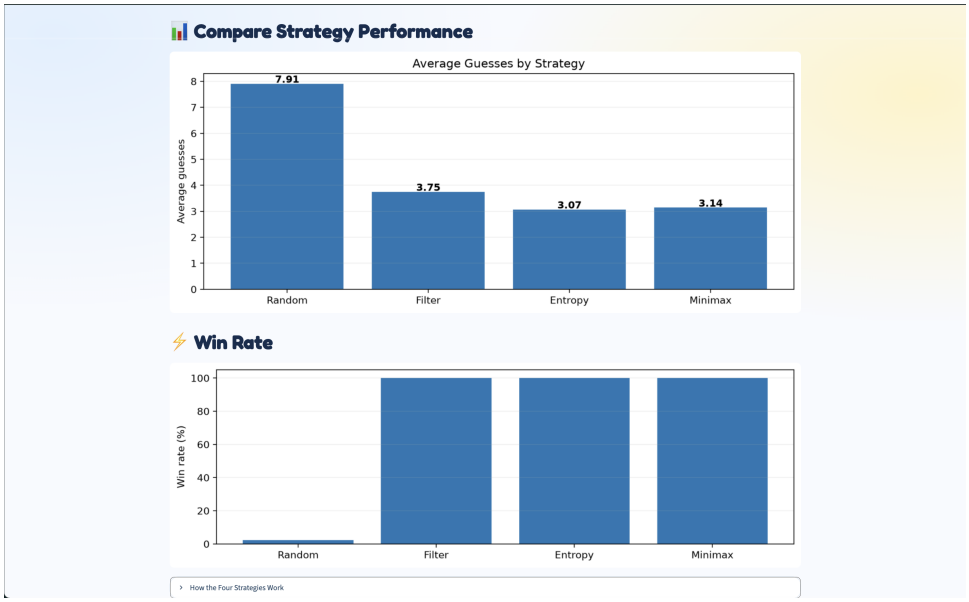


Figure 3: Algorithm Comparison Interface

6. Experimental Setup

All four strategies were evaluated using the same dataset of 386 Pokémon records and a maximum of eight guesses per game.

The evaluation procedure used different run counts for different strategies:

- **Random:** Five runs for each of the 386 possible hidden Pokémon.
- **Filter:** Five runs for each of the 386 possible hidden Pokémon.
- **Entropy:** One run for each of the 386 possible hidden Pokémon.
- **Minimax:** One run for each of the 386 possible hidden Pokémon.

Therefore, Random and Filter each completed 1,930 runs, while Entropy and Minimax each completed 386 runs.

The following performance measures were recorded:

- Average guesses required per game.
- Median number of guesses.
- Worst-case score.
- Standard deviation.
- Win rate.
- Percentage of games solved within six guesses.
- Average execution time per game in milliseconds.

A successful game is scored using the number of guesses required. An unsuccessful game receives a score of 9, according to the project's `FAIL_SCORE` convention. The actual game limit remains eight guesses.

7. Results and Performance Comparison

The benchmark results show differences in the efficiency and computational cost of the four guessing strategies.

7.1 Benchmark Results

Table 2: Performance Comparison of Guessing Strategies

Strategy	Runs	Avg.	Median	Worst	Std. Dev.	Win (%)	≤6 (%)	ms/game
Random	1930	8.91	9	9	0.71	1.97	1.45	0.08
Filter	1930	3.67	4	9	1.09	99.95	99.12	0.61
Entropy	386	3.07	3	5	0.63	100.00	100.00	2.72
Minimax	386	3.14	3	5	0.64	100.00	100.00	2.29

Avg. = Average guesses. A lost game receives a score of 9 (`FAIL_SCORE`); the actual game limit is eight guesses.

7.2 Benchmark Visualization

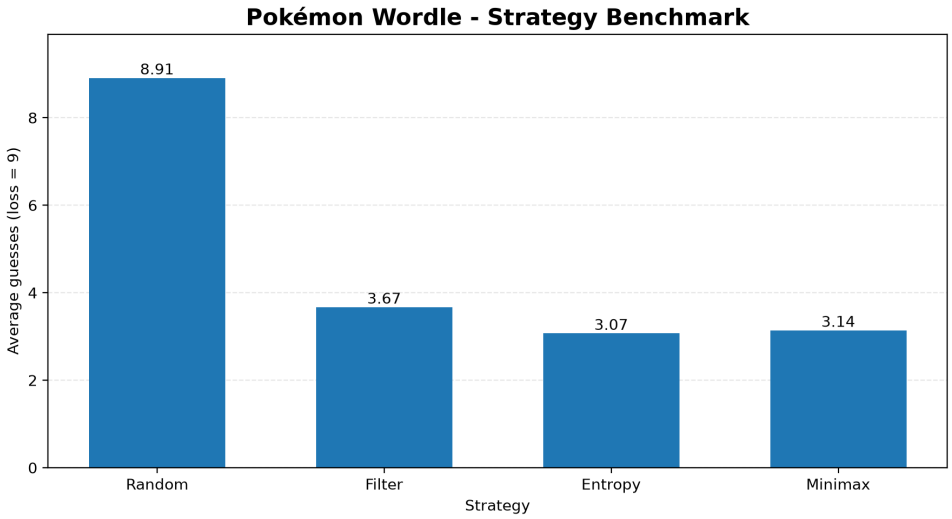


Figure 4: Comparison of Guessing Strategy Performance

7.3 Results Interpretation

The benchmark results reveal differences in the performance of the four guessing strategies.

- **Random:** Achieved an average score of 8.91 and a win rate of 1.97%, reflecting its lack of feedback-based decision-making.
- **Filter:** Achieved an average of 3.67 guesses and a win rate of 99.95%. It solved 99.12% of games within six guesses.
- **Entropy:** Achieved the lowest average of 3.07 guesses and solved all 386 evaluated cases, with a worst-case result of five guesses.
- **Minimax:** Achieved an average of 3.14 guesses and solved all 386 evaluated cases, also with a worst-case result of five guesses.

Entropy achieved the lowest average guess count, while Minimax produced comparable results. Filter also demonstrated strong performance with lower computational time than the advanced strategies.

8. Discussion

The benchmark comparison highlights the trade-off between guessing efficiency and computational cost.

Random guessing performed poorly because it does not use previous feedback to guide subsequent guesses. Filter substantially improved performance by eliminating candidates that were inconsistent with the observed clues.

Entropy achieved the lowest average guess count, while Minimax provided comparable performance. Both advanced strategies solved all evaluated cases within five guesses.

Filter required less computational time per game than Entropy and Minimax. This demonstrates that candidate elimination can provide an effective balance between computational cost and guessing efficiency.

The benchmark used different run counts for the strategies. Random and Filter were evaluated five times for each of the 386 possible hidden Pokémon, resulting in 1,930 runs each. Entropy and Minimax were evaluated once for each hidden Pokémon, resulting in 386 runs each.

All games used a maximum of eight guesses. Unsuccessful games were assigned a benchmark score of nine using the project's `FAIL_SCORE` convention.

9. Conclusion

Pokémon Wordle demonstrates how different algorithmic approaches can be applied to an attribute-based guessing game.

The project brings together an interactive Streamlit application, modular game implementation, automated solvers, and a benchmarking framework.

Among the evaluated strategies, Entropy achieved the lowest average number of guesses at 3.07. Minimax achieved comparable performance with an average of 3.14 guesses and a worst-case result of five guesses.

Filter offered an effective candidate-elimination approach with a low computational cost, whereas Random served as a baseline for comparison.

Overall, the project illustrates the practical value of candidate elimination, information theory, and worst-case optimization in intelligent decision-making.

10. References

1. Streamlit Documentation. https://docs.streamlit.io/
2. Python Documentation. https://docs.python.org/3/
3. pandas Documentation. https://pandas.pydata.org/docs/
4. scikit-learn Documentation. https://scikit-learn.org/stable/
5. Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379–423.